Universidade Federal do ABC

Controlador Neural de Drones Quadricópteros desenvolvido por meio de Algoritmos Genéticos

Relatório PGC III

Nome:	*RA:*
Ivan Zanutto Bastos	11007116
Matheus Pacheco de Oliveira	11201722352

Orientador: Hugo Puertas de Araújo

Abril/2025

Resumo

O avanço tecnológico tem impulsionado o uso de drones autônomos em diversas áreas, destacando-se aplicações como agricultura de precisão, logística e gerenciamento de desastres, onde esses dispositivos oferecem maior eficiência e escalabilidade em comparação às operações humanas. Neste contexto, redes neurais artificiais surgem como uma solução promissora devido à sua capacidade de adaptação a ambientes dinâmicos e não lineares. Paralelamente, algoritmos genéticos (AGs) têm se mostrado eficazes na otimização de redes neurais, explorando de forma eficiente o espaço de soluções para ajustar os pesos e parâmetros dessas redes.

Este trabalho propõe o desenvolvimento de um controlador neural para drones quadricópteros utilizando AGs, com foco na navegação autônoma em ambientes simulados. O controlador será treinado para deslocar o drone até locais designados, assumindo que o voo ocorre continuamente, sem considerar decolagem ou pouso. Para isso, foi modelado um ambiente virtual utilizando a plataforma Gymnasium, implementada uma rede neural para controle do drone e implementados algoritmos genéticos para otimização do controlador.

Abstract

The rapid technological advancements have driven the adoption of autonomous drones across various fields, notably in precision agriculture, logistics, and disaster management, where these devices provide increased efficiency and scalability compared to human-operated systems. In this context, artificial neural networks emerge as a promising solution due to their ability to adapt to dynamic and nonlinear environments. Simultaneously, genetic algorithms (GAs) have proven effective in optimizing neural networks by efficiently exploring solution spaces to adjust weights and parameters.

This study proposes the development of a neural controller for quadcopter drones using GAs, focusing on autonomous navigation in simulated environments. The controller is trained to navigate the drone to designated locations, assuming continuous flight without addressing takeoff or landing. To achieve this, a virtual environment was modeled using the Gymnasium platform, a neural network was implemented for drone control, and genetic algorithms were applied to optimize the controller.

1 Introdução

Nos últimos anos, tem-se observado um crescimento significativo nas pesquisas relacionadas ao desenvolvimento e aplicação de drones em diversas áreas, com destaque para o uso de drones autônomos. Esses dispositivos têm se mostrado particularmente promissores, pois oferecem uma redução substancial nos custos operacionais e uma maior escalabilidade em comparação com operações que requerem intervenção humana. Dentre as principais áreas de aplicação, destacam-se a agricultura de precisão, onde os drones são utilizados para monitoramento de culturas e pulverização de insumos; os serviços de entrega, que prometem maior eficiência logística ao alcançar áreas de difícil acesso; e o gerenciamento de desastres, onde a rapidez e a precisão dos drones são essenciais para mapear áreas afetadas, auxiliar buscas e salvamentos, além de avaliar danos (**??????**).

Nesse contexto, as redes neurais artificiais emergem como uma solução promissora devido à sua capacidade de aprender e generalizar comportamentos complexos a partir de dados de treino, sem a necessidade de modelos explícitos do sistema. Essas redes, inspiradas no funcionamento do cérebro humano, são particularmente eficazes em ambientes dinâmicos e não lineares, onde as variáveis e condições de operação podem mudar rapidamente. Essas características tornam as redes neurais uma ferramenta poderosa para o desenvolvimento de controladores de voo que precisam se adaptar continuamente às variáveis do ambiente, como ventos ou obstáculos inesperados (**??**).

Paralelamente, o uso de algoritmos genéticos (AGs) tem se mostrado uma abordagem promissora para realizar o treinamento de redes neurais. Inspirados nos processos de seleção natural, os AGs são ferramentas poderosas para a otimização de parâmetros em sistemas complexos, como aqueles envolvidos no ajuste dos pesos de uma rede neural. Eles permitem a exploração eficiente do espaço de soluções, encontrando configurações de rede que maximizam o desempenho do drone em diferentes cenários operacionais (**????**).

Sendo assim, este trabalho propõe a combinação de redes neurais e algoritmos genéticos como uma solução robusta e adaptável para o desenvolvimento de um controlador de voo de drones autônomos, com potencial para superar desafios presentes em aplicações críticas.

2 Objetivos

2.1 Objetivo Geral:

Desenvolver um controlador neural para drones quadricópteros utilizando Algoritmos Genéticos, capaz de realizar navegação autônoma em um ambiente virtual, deslocando-se até um local designado e evitando obstáculos. O controlador assumirá que o drone permanecerá em voo durante todo o trajeto, isto é, não incluirá pouso nem decolagem.

2.2 Objetivos Específicos:

1. Desenvolver um Ambiente Virtual Adequado: Modelar e implementar um ambiente virtual baseado em Gymnasium para que seja possível reproduzir e visualizar o comportamento do drone.

2. Desenvolver uma Rede Neural: Projetar e implementar uma rede neural artificial para controlar o comportamento do drone no ambiente simulado.

3. Aplicar Algoritmos Genéticos: Utilizar AGs para otimizar a rede neural, ajustando os parâmetros e a arquitetura da rede para melhorar o desempenho do controlador.

4. Simular e Avaliar: Implementar o controlador neural em um ambiente virtual, avaliando sua eficácia em termos de navegação e capacidade de evitar obstáculos.

3 Proposta

O projeto começará pela modelagem do ambiente simulado no *Gymnasium*, uma biblioteca *open source* Python. Esse ambiente será em 3 dimensões e configurado para incluir uma série de obstáculos e destinos para o drone, definindo também os parâmetros de qualidade do modelo para calcular nossa *fitness function* (função de aptidão), oferecendo um cenário de teste robusto para o controlador.

A rede neural será inicialmente projetada com uma arquitetura simples, ajustada para lidar com o controle em tempo real do drone. A estrutura da rede incluirá entradas correspondentes a sensores do drone (como distâncias até obstáculos e posição relativa

ao destino) e saídas que corresponderão aos comandos de controle, como ativação e velocidade das hélices.

Os Algoritmos Genéticos serão utilizados para treinar a rede neural, começando com uma população inicial de redes com parâmetros aleatórios. O desempenho de cada indivíduo será avaliado com base na capacidade do drone de atingir o destino, ou o quão perto ele alcançou, sem colidir com obstáculos. Os melhores indivíduos serão selecionados para reprodução, aplicando operadores genéticos como cruzamento e mutação, para gerar a próxima geração de redes neurais. Este processo será repetido até que o controlador atinja um desempenho satisfatório, ou até que um número máximo de gerações seja atingido.

Finalmente, o desempenho do controlador treinado será avaliado em uma série de testes simulados. Será verificado o sucesso do drone em diferentes configurações de ambiente.

4 Metodologia

4.1 Ambiente Simulado

Para o desenvolvimento do ambiente, foi utilizado como base o projeto *gym-pybullet-drones* (**??**), que utiliza a biblioteca *PyBullet*(**??**) para a simulação da física e o *Crazyflie 2.0* (**??**) como modelo de drone. Abaixo estão detalhadas as principais propriedades do ambiente e as alterações feitas para adequá-lo aos requisitos do trabalho.

4.1.1 Espaço de Observação

O espaço de observação da rede neural foi definido como: Posição relativa ao destino (via GPS), velocidade (via GPS), aceleração (via acelerômetro), quatérnio de orientação (via giroscópio), mapa de profundidade (via LIDAR) e altura (via sensor de distância).

4.1.2 Espaço de Ação

O espaço de ação consiste em 4 valores que variam de -1 a 1 e que controlam a rotação de cada motor individualmente conforme a função:

$$rpm(x) = h(1 + 0,05x)$$

Em que h é a rotação necessária para manter o drone parado no ar.

4.1.3 Forças não Conservativas

O *PyBullet* possui uma propriedade chamada *damping*, que atua como atrito global para todos os objetos da simulação. Ela funciona da seguinte forma:

$$a_d = -d_v v, \text{ e } \alpha_d = -d_\omega \omega$$

Em que a_d é a aceleração linear causada pelo *damping*, d_v o *damping* linear e v a velocidade linear. E em seguida estão os seus análogos angulares.
Por padrão, $d_v = d_\omega = 0,04$, porém, o projeto base *gym-pybullet-drones* possui uma implementação de arrasto baseada em dados experimentais, que foi utilizada em vez do *damping* linear. O *damping* angular não foi alterado.

4.1.4 Critério de Conclusão e Truncagem

O ambiente é considerado concluído se a distância do drone até o objetivo chegar ao limiar de 0,5 metros.
O ambiente é truncado caso a inclinação do drone ultrapasse o limiar de 0,4 radianos ou a distância até o objetivo ultrapasse o limiar de $\sqrt{2}$ vezes a distância inicial.

4.1.5 Função de Fitness

A princípio, foi utilizada apenas a distância em relação ao objetivo como atributo da função de *fitness*, que era dada por:

$$F_d(d) = \begin{cases} 1 - (d/D)^2, & \text{se } d \leq D \\ 0, & \text{caso contrário} \end{cases}$$

Sendo d a distância final e D a distância inicial

No entanto, para garantir que o drone irá parar no objetivo, e para incentivar a eficiência do trajeto, foram introduzidas a velocidade final e a duração total como atributos.

Sendo assim, a função final utilizada é dada por:

$$F(d,v,t) = \begin{cases} 50 \times F_d(d), & \text{se } d \geq \frac{\sqrt{2}}{2} \\ 50 \times F_d(d) + 30 \times F_v(v), & \text{se } d < \frac{\sqrt{2}}{2} \text{ e se } v \geq 1 \\ 50 \times F_d(d) + 30 \times F_v(v) + 20 \times F_t(t), & \text{caso contrário} \end{cases}$$

Em que FR_v e F_t são dadas por:

$$F_v(v) = 1 - (v/V)^2$$

Sendo v a velocidade final e V a velocidade máxima

$$F_t(t) = 1 - \left(\frac{t - T_{min}}{T_{max} - T_{min}}\right)^2$$

Sendo t = tempo final, $T_{max} = \frac{\text{distância inicial}}{\text{velocidade mínima}}$ e $T_{min} = \frac{\text{distância inicial}}{\text{velocidade máxima}}$
A velocidade mínima é definida como 1 m/s, e a velocidade máxima como 20 m/s

Esta definição faz com que o *fitness* funcione por patamares, ou seja, a velocidade só passa a ser relevante quando a distância é menor que $\frac{\sqrt{2}}{2}$, e o tempo só passa a ser relevante quando a velocidade é menor que 1. Os valores foram escolhidos com base no limiar de conclusão $(0,5)$. A ideia por trás desta abordagem é colocar uma ordem de prioridade nos atributos, o que evita uma competição entre eles, acelerando a convergência.

Além disso, a função agora vai de 0 a 100, ao invés de 0 a 1, para facilitar a visualização, sendo destes 50 relacionados à distância, 30 relacionados à velocidade e 20 relacionados ao tempo. Estes valores foram escolhidos com base na importância de cada atributo em relação ao resultado desejado.

4.2 Algoritmos Genéticos - Utilizando DEAP

Inicialmente, foram feitos testes utilizando a biblioteca PyGAD (*Python Genetic Algorithm* (**??**) para a implementação de Algoritmos Genéticos, porém, após uma análise mais profunda, foi decidida a utilização da biblioteca DEAP (*Distributed Evolutionary Algorithms in Python*) (**??**), por permitir uma maior flexibilidade e complexidade nos algoritmos que forem utilizados.

4.2.1 Hiperparâmetros

Para este algoritmo, primeiro é necessário definir alguns hiperparâmetros, que posteriormente serão explorados para a otimização da aplicação. São eles:

- Mínimo de camadas ocultas
- Máximo de camadas ocultas
- Mínimo de neurônios por camada oculta
- Máximo de neurônios por camada oculta
- Número de indivíduos na população
- Quantidade de gerações
- Taxa de cruzamento
- Taxas de mutação
- Tamanho do torneio de seleção

4.2.2 Representação do Indivíduo

O cromossomo foi definido de tamanho fixo em função dos hiperparâmetros de quantidades máxima e mínima das camadas ocultas. Para isso, é necessário que exista a possibilidade de alguns genes estarem inativos de acordo com a configuração da topologia de cada indivíduo. A funcionalidade de cada gene é a seguinte:

- Índice 0: Quantidade de camadas ocultas (respeitando os limites máximos e mínimos dos hiperparâmetros)
- Índices i=1 a i=*quantidade máxima de neurônios*: quantidade de neurônios da camada i (respeitando os limites máximos e mínimos dos hiperparâmetros)
- Todos os genes seguintes representam os pesos sinápticos da rede neural. Cada camada possui índices pré-determinados, definidos como o próximo índice disponível até a quantidade máxima de pesos por camada, calculada a partir da quantidade máxima de neurônios definida em hiperparâmetro.

4.2.3 Algoritmo Utilizado

O primeiro algoritmo utilizado foi o *deap.algorithms.eaSimple()*, que reproduz o algoritmo evolucionário mais simples. A cada geração, cada indivíduo é avaliado conforme a recompensa gerada pelo ambiente simulado e então é feita a seleção por torneio, e aplicados os operadores genéticos tanto de cruzamento quanto de mutação. Com esse algoritmo, os genes dos indivíduos foram unificados para um indivíduo ter tanto os genes da evolução topológica quanto os dos pesos da rede neural.

Um segundo teste feito a partir desse algoritmo foi dividir a população em duas:

1. População de evolução topológica. Utilizando o melhor indivíduo da população de pesos de forma fixada.

2. População de evolução dos pesos da rede neural. Utilizando o melhor indivíduo da população de topologia de forma fixada.

Dessa forma, a cada macro-geração do algoritmo, a população topológica é evoluída por 3 gerações usando os pesos com melhor *fitness* e depois a população de pesos é evoluída por 3 gerações usando a melhor topologia. Dessa forma, os dois problemas, a topologia e os pesos sinápticos, serão evoluídos de forma independente, se aproveitando de uma coevolução mutualística.

4.3 Algoritmos Genéticos - Utilizando NEAT

Por fim, também foi utilizada a biblioteca *neat-python* (**??**) que implementa o algoritmo *NeuroEvolution of Augmenting Topologies* (NEAT) (**??**). Esse algoritmo se baseia na ideia de iniciar a população com a topologia mais simples possível, normalmente apenas com a camada de entrada e a de saída, e adicionar neurônios e conexões através de mutação ao passar das gerações.

4.3.1 Representação do Indivíduo

Como proposto pelo artigo original, cada gene pode representar um nó (neurônio) ou uma conexão, e terão alguns atributos:

1. Número de inovação, para registrar a origem histórica desse gene

2. Peso sináptico

3. Se habilitado ou desabilitado

4. Se conexão: os dois nós conectados

5. Se nó: a função de ativação

4.3.2 Mutação

A mutação pode acontecer a cada geração das seguintes formas:

1. Perturbação dos pesos sinápticos

2. Adição de conexão

3. Adição de nó

4. Mudança da função de ativação (com exceção da camada de saída)

5. Desabilitar ou habilitar determinada conexão

Ao adicionar um nó ou uma conexão, é atribuído um número de inovação para esse gene. Este número é um inteiro incremental, ou seja, o primeiro nó (ou conexão) adicionado na população terá número 1, o segundo terá número 2, e assim sucessivamente. Ele representa a origem histórica desse gene, o que torna possível distinguir e comparar genes de organismos diferentes, já que genes com o mesmo número possuem a mesma origem e, portanto, representam o mesmo nó (ou conexão). Isso será muito útil e utilizado para o processo de *crossover* e especiação.

4.3.3 Crossover

Para fazer o *crossover* entre dois organismos diferentes (pais), primeiro são comparados seus genes de acordo com o número de inovação e então cada gene é classificado em uma das três seguintes categorias:

1. **Genes correspondentes (*matching*):** São aqueles que aparecem em ambos os pais e compartilham o mesmo número de inovação.

2. **Genes disjuntos (*disjoint*):** São genes cujo número de inovação existe em apenas um dos pais, mas cujo valor é menor ou igual ao maior número de inovação presente no outro pai. Esses genes ficam "entre" as inovações concordantes dos dois pais e representam diferenças em regiões comparáveis da linha evolutiva.

3. **Genes em excesso (*excess*):** São genes exclusivos de um dos pais cujo número de inovação é maior do que o maior número de inovação encontrado no outro pai. Esses genes representam características evolutivas mais recentes de um dos organismos.

Finalmente, o cruzamento é feito da seguinte forma:

- *Genes correspondentes* têm peso herdado de forma aleatória entre os dois pais.
- *Genes disjuntos* e *em excesso* são copiados diretamente para o filho a partir do indivíduo mais apto (ou de ambos, caso haja empate de aptidão).

4.3.4 Especiação

Esse algoritmo também adiciona a ideia de **especiação** para proteger novas inovações, já que um novo nó pode, por exemplo, piorar o *fitness* a curto prazo, mas ser útil a longo prazo depois de evoluir seu peso por gerações.

Para isso, é introduzida a ideia da distância de compatibilidade, δ, entre dois genomas diferentes, que representa o quão diferentes são esses dois organismos. Caso essa distância esteja acima de certo limiar, δ_t, os dois organismos são considerados de espécies diferentes.

$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 \overline{W}$$

Onde,

E: Quantidade de genes em excesso

D: Quantidade de genes disjuntos

$\overline{W}$: Diferença média dos pesos sinápticos

N: Tamanho do maior genoma

c_1, c_2, c_3: Coeficientes parametrizáveis

Para proteger novas inovações, o *fitness* de cada organismo é ajustado ao ser dividido pelo tamanho de sua espécie. Assim, penaliza-se espécies grandes e impossibilita que uma única espécie domine toda a população, além de dar tempo para que novas mutações se desenvolvam.

Além disso, espécies são inteiramente extintas caso não melhorem em determinado tempo (neste caso, 15 gerações), com exceção das 3 melhores espécies que são protegidas por elitismo para evitar a erradicação total da população.

Por fim, para alocar descendentes, primeiro é atribuído um número de descendentes para cada espécie proporcional à soma do *fitness* ajustado de todos os seus organismos. Então, são removidos os organismos de pior desempenho e os restantes se reproduzem. Toda a população de cada espécie é trocada por seus descendentes, com exceção dos organismos protegidos por elitismo (2 por espécie).

5 Resultados

Os primeiros resultados obtidos foram com base na posição-alvo do drone fixa durante toda a evolução e serão divididos entre 3 algoritmos:

1. Utilizando DEAP - População única
2. Utilizando DEAP - Coevolução com população dividida entre evolução da topologia e dos pesos sinápticos
3. Utilizando NEAT

Em todos os casos, a camada inicial possui 14 neurônios para se adequar ao espaço de observação do ambiente simulado, e a camada de saída possui 4 neurônios com a função de ativação de tangente hiperbólica, cada um representando um motor do drone, que por sua vez pode assumir valores de -1 a 1.

Porém, entende-se a necessidade de uma maior exploração de hiperparâmetros, e para isso, a necessidade de um maior poder computacional para suportar tais testes.

5.1 Utilizando DEAP - População Única

Como pode ser observado na Figura **??**, o primeiro algoritmo testado demonstrou a viabilidade da solução por evolução, melhorando o fitness médio a cada geração e ocasionalmente melhorando o melhor fitness a depender de alguma mutação. Porém, não foi capaz de encontrar a solução ótima já que o maior fitness teve valor de 49,3, ou seja, chegou perto de encostar no alvo, mas não o suficiente. Na Figura **??** é possível ver uma convergência prematura com os cromossomos evoluindo para 1 camada oculta com 15 neurônios.

Esse algoritmo foi executado com tamanho de população de 50 indivíduos. Cada rede pode ter de 1 a 10 camadas ocultas e cada camada oculta pode ter de 5 a 20 neurônios. A cada mutação tem 10% de chance de aumentar ou diminuir a quantidade de camadas, 10% de chance de mudar a quantidade de neurônios de uma camada aleatória e pesos sinápticos são perturbados com base em uma distribuição normal.

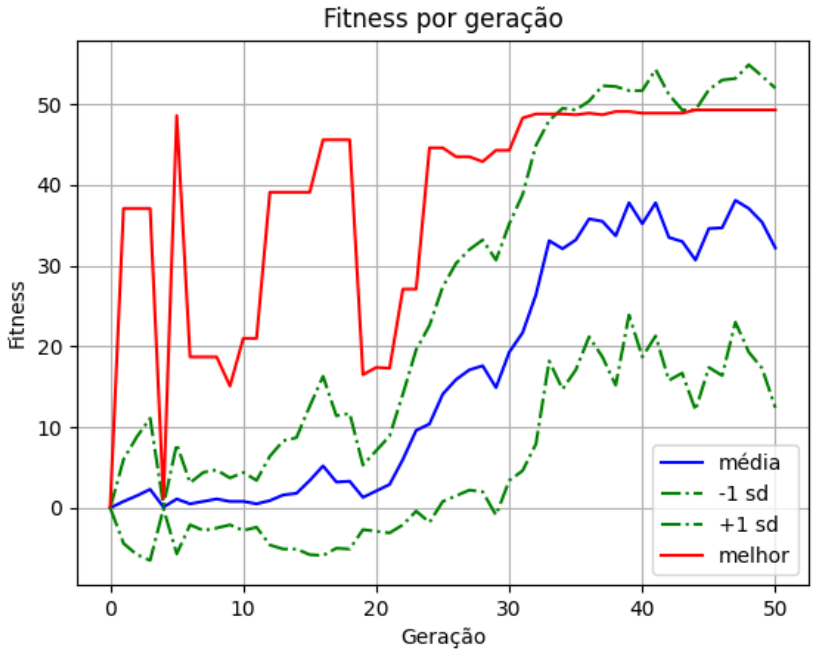
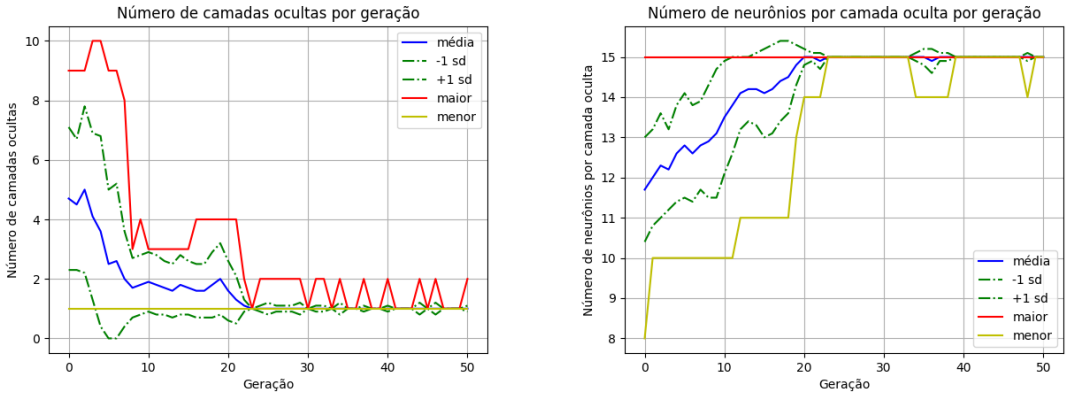


Figura 1: Desempenho do primeiro algoritmo com base no fitness por geração.

5.2 Utilizando DEAP - Coevolução

Na Figura **??** pode ser observada a evolução do fitness desse algoritmo coevolucionário (evoluindo separadamente a topologia da rede e os pesos sinápticos), que encontrou o melhor fitness de 58,2 e média de 34,9. O algoritmo foi executado de forma que cada população tenha 500 indivíduos e 100 macro-gerações, que por sua vez se dividem em 3 micro-gerações de evolução da topologia e 3 micro-gerações de evolução dos pesos sinápticos. Também foi implementado o processo de elitismo, no qual 10% dos melhores pais necessariamente continuam para a próxima geração.



((a)) Evolução da quantidade de camadas ocultas

((b)) Evolução da quantidade de neurônios por camada oculta

Figura 2: Evolução da topologia da rede utilizando o primeiro algoritmo.

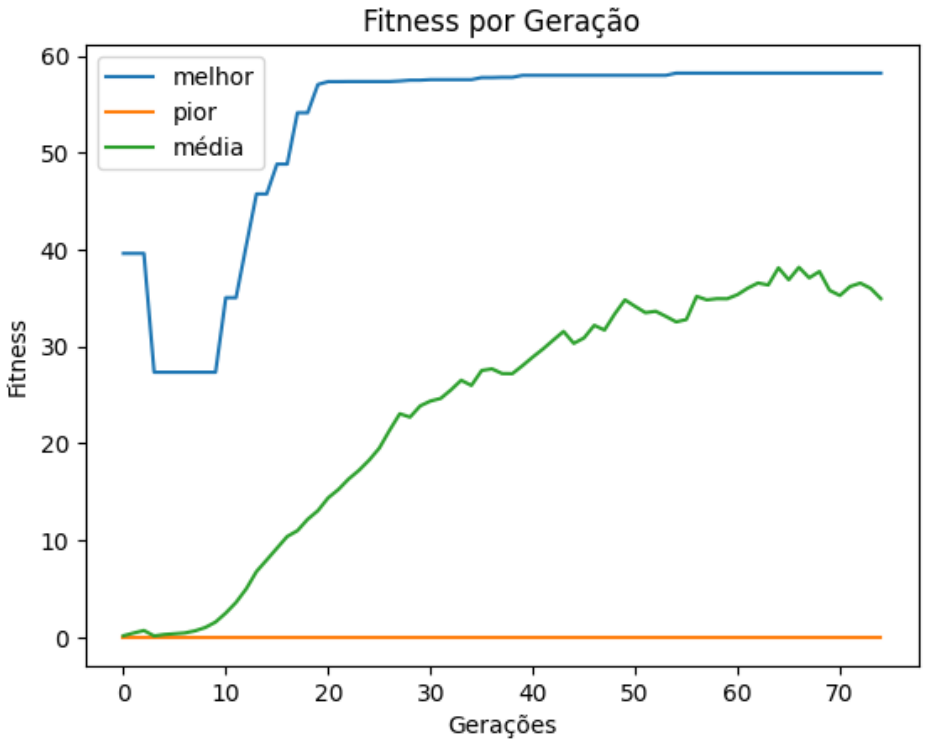


Figura 3: Desempenho do algoritmo com base no fitness por geração.

5.3 Utilizando NEAT

Por fazer uma implementação nova de rede neural, o algoritmo com o NEAT foi mais rápido de rodar, já que não usa a biblioteca *keras* e nem a *pytorch* que adicionam *overhead* na execução do algoritmo. Com isso, foi possível uma maior exploração de hiperpârametros e um tamanho maior da população.

Assim, o algoritmo foi executado com o tamanho da população de 1500 indivíduos, as opções de função de ativação foram *sigmoid, relu* ou *tanh*.

Na Figura **??** é possível ver a evolução do fitness ao longo das gerações encontrando fitness máximo de 98,49, ou seja, o drone conseguiu chegar e parar na localização alvo com o único ponto de melhoria sendo o tempo para alcançar o resultado, mas já muito próximo do fitness teórico máximo (100). A Figura **??** representa a mesma visualização, mas mostrando apenas as primeiras 100 gerações. Com isso, pode-se verificar que acontece uma grande evolução até por volta da geração 20, e depois o algoritmo necessita de mutações esporádicas para alcançar fitness melhores.

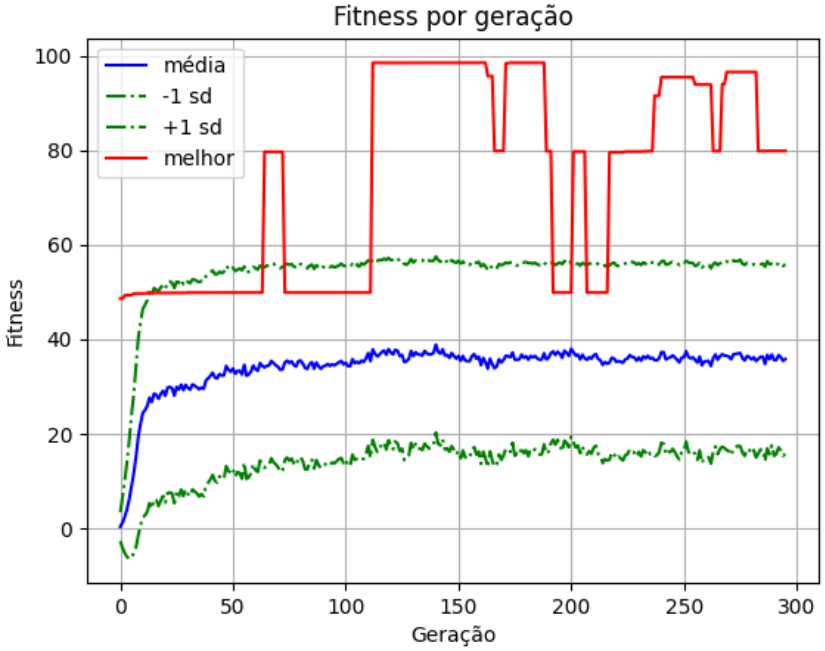


Figura 4: Desempenho do algoritmo NEAT com base no fitness por geração.

Na Figura **??** é possível visualizar como ocorreu o processo de especiação na população. Sendo que a Figura **??** demonstra o tamanho das espécies, ou seja, a quantidade máxima, mínima e média de organismos por espécie ao longo da evolução, en-

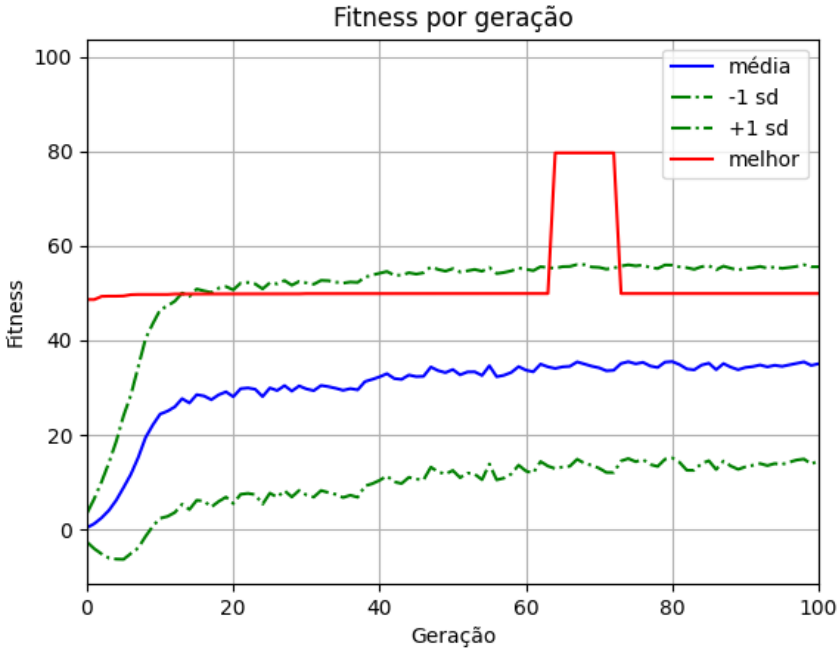
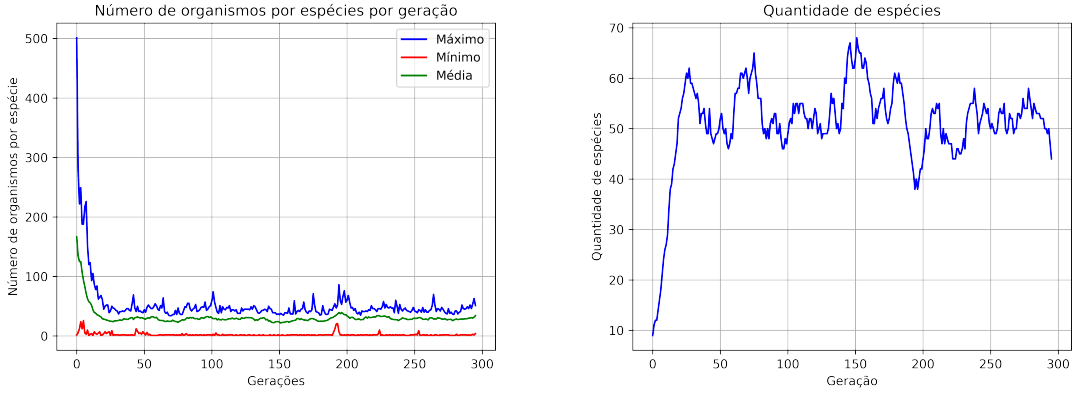


Figura 5: Evolução de fitness do algoritmo NEAT nas primeiras 100 gerações.

quanto a Figura **??** demonstra a quantidade de espécies a cada geração.



((a)) Tamanho das espécies por geração. ((b)) Quantidade de espécies por geração.

Figura 6: Evolução da especiação da população.

Por fim, na Figura **??** é demonstrada a rede que teve o maior fitness, sendo que os nós com forma quadrada representam neurônios de entrada, já os nós redondos com índices de 0 a 3 são nós de entrada e, finalmente, os nós redondos remanescentes são neurônios de camadas intermediárias. As conexões de cor verde representam pesos positivos,

enquanto as de cor vermelha representam pesos negativos. Ainda sobre as conexões entre nós, as tracejadas representam conexões desabilitadas. Portanto, é possível ver que a rede neural vencedora (com o maior fitness) teve dois neurônios intermediários com função de ativação *relu*.

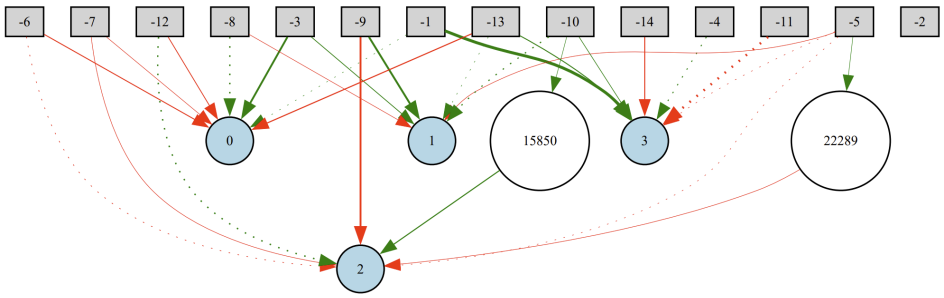


Figura 7: Arquitetura da rede neural vencedora gerada pelo algoritmo NEAT.

6 Cronograma

Etapas	2024							2025				
	Jun	Jul	Ago	Set	Out	Nov	Dez	Jan	Fev	Mar	Abr	Mai
Revisão Bibliográfica	X	X	X	X	X							
Desenvolvimento da Metodologia		X	X	X								
Implementação do Ambiente				X	X	X	X					
Implementação dos Algoritmos				X	X	X	X	X				
Análise dos Resultados								X	X			
Redação do Relatório Final									X	X	X	
Revisão e Ajustes												X

Referências

BENARBIA, T.; KYAMAKYA, K. A literature review of drone-based package delivery logistics systems and their implementation feasibility. *Sustainability (Switzerland)*, v. 14, 2022. ISSN 20711050.

COUMANS, E.; BAI, Y. *PyBullet, a Python module for physics simulation for games, robotics and machine learning*. 2016–2019. ⟨http://pybullet.org⟩.

DAUD, S. M. S. M. et al. Applications of drone in disaster management: A scoping review. *Science and Justice*, v. 62, 2022. ISSN 18764452.

FÖRSTER, J. *System identification of the crazyflie 2.0 nano quadrocopter*. Dissertação (B.S. thesis) — ETH Zurich, 2015.

FORTIN, F.-A. et al. DEAP: Evolutionary algorithms made easy. *Journal of Machine Learning Research*, v. 13, p. 2171–2175, jul 2012.

GAD, A. F. Pygad: An intuitive genetic algorithm python library. *CoRR*, abs/2106.06158, 2021. Disponível em: ⟨https://arxiv.org/abs/2106.06158⟩.

GEN, R. C. M. *Genetic Algorithms and Engineering Design*. 1. ed. [S.l.]: Wiley-Interscience, 1997. (Engineering Design and Automation). ISBN 0471127418; 9780471127413.

HAQUE, A.; CHOWDHURY, M. N.-U.-R. Exploring the benefits of reinforcement learning for autonomous drone navigation and control. *International Journal of Advanced Networking and Applications*, v. 15, 2023. ISSN 09750282.

KATOCH, S.; CHAUHAN, S. S.; KUMAR, V. A review on genetic algorithm: past, present, and future. *Multimedia Tools and Applications*, v. 80, 2021. ISSN 15737721.

MCINTYRE, A. et al. *neat-python*.

PANERATI, J. et al. Learning to fly—a gym environment with pybullet physics for reinforcement learning of multi-agent quadcopter control. In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. [S.l.: s.n.], 2021.

REJEB, A. et al. Drones in agriculture: A review and bibliometric analysis. *Computers and Electronics in Agriculture*, v. 198, 2022. ISSN 01681699.

STANLEY, K. O.; MIIKKULAINEN, R. Evolving neural networks through augmenting topologies. *Evolutionary computation*, MIT Press, v. 10, n. 2, p. 99–127, 2002.